Project 1

Tătaru Mihai-Radu, Group 407 AI

June 1, 2025

Problem 1

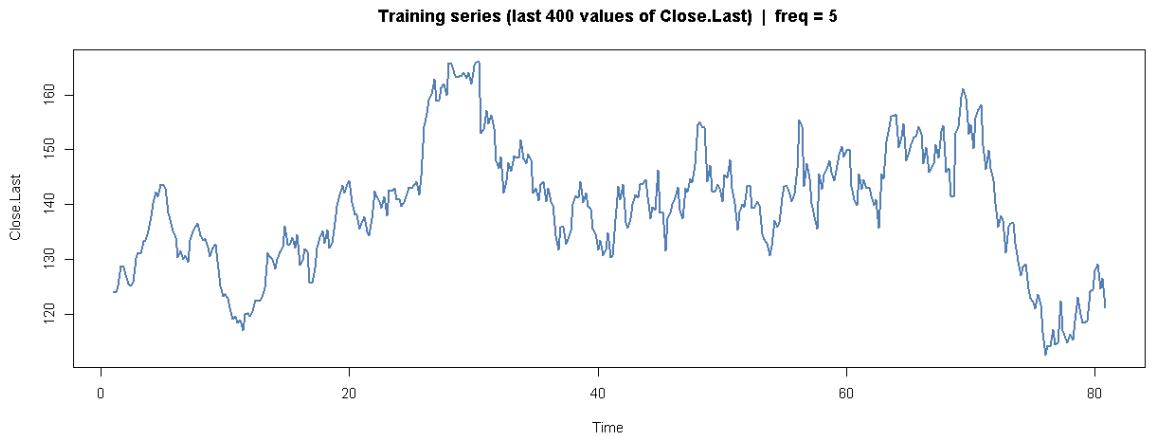
This task is divided in the following 5 steps:

1. Analysing the data series
2. Transformations for stationarity
3. Finding a proper model
4. Diagnostics
5. Forecasting

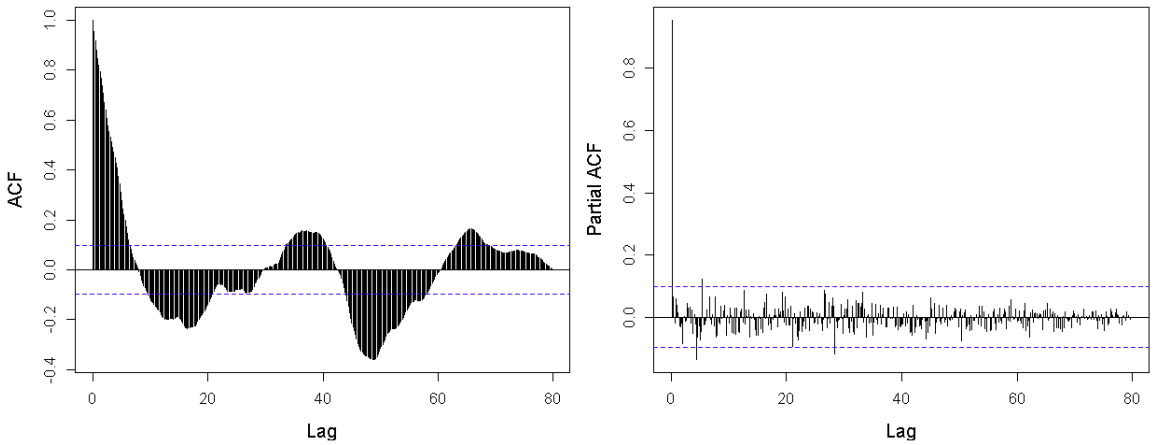
1.1 Analysing the data series

The time series I used is the stock price of "PHLX Gold/Silver Sector" which contains information about the price of stock from the last 420 working days (13 May was the latest when I downloaded the .csv). I use the column **Close/Last** and divide the data into train (values from day 21 to day 420) and test (day 1 to day 20), then convert it into a time series object with frequency = 5 (the frequency is needed for when I use the *auto_arima* function and check for sesonality). So each unit of "time" equals 5 days.

Below you can find the plots for the time series (*Figure* 1.1a), the ACF and PACF (*Figure* 1.1b). The ACF plot looks very weird, there might be some kind of sesonality, and the PACF plot cuts off after lag 1. Further analysis is needed to draw some conclusions.



(a) Original TS



(b) ACF and PACF plots for the Original TS

Figure 1.1

Now I perform the **Augmented Dickey-Fuller** and **KPSS** tests (*Figure* 1.2a) to check for stationarity. To be sure that the time series is stationary, I need to the p-value for ADF to be smaller than 0.05 and the p-values for the KPSS tests greater than 0.05. The oposite happens, so I conclude that the series is not stationary. I need to some transformations to make it stationary.

```
        Augmented Dickey-Fuller Test

data:  ts_data
Dickey-Fuller = -2.1297, Lag order = 7, p-value = 0.5225
alternative hypothesis: stationary


        KPSS Test for Level Stationarity

data:  ts_data
KPSS Level = 0.65879, Truncation lag parameter = 5, p-value = 0.01729


        KPSS Test for Trend Stationarity

data:  ts_data
KPSS Trend = 0.58124, Truncation lag parameter = 5, p-value = 0.01
```

(a) Stationarity tests for Original TS

```
Estimated lambda: -0.4129

Stationarity tests on Box-Cox-transformed series:

        Augmented Dickey-Fuller Test

data:  ts_data
Dickey-Fuller = -2.1203, Lag order = 7, p-value = 0.5265
alternative hypothesis: stationary


        KPSS Test for Level Stationarity

data:  ts_data
KPSS Level = 0.67301, Truncation lag parameter = 5, p-value = 0.016


        KPSS Test for Trend Stationarity

data:  ts_data
KPSS Trend = 0.61021, Truncation lag parameter = 5, p-value = 0.01
```

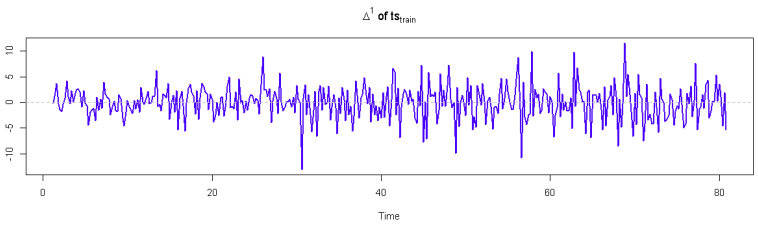
(b) Stationarity tests for BoxCox TS

Figure 1.2

1.2 Transformations for stationarity

I try to apply a **Box-Cox** transformation to the time series using the BoxCox function implemented in R, which also figures out the appropriate λ to use. Sadly, I do not obtain stationarity after this transformation (*Figure* 1.2b).

Next, I differentiate the time series once and this time I obtain stationarity! *Figure* 1.3a is the plot of the time series obtained and *Figure* 1.3b shows the stationarity tests.



(a) Stationarity tests for Original TS

```
        Augmented Dickey-Fuller Test

data:  ts_data
Dickey-Fuller = -7.6307, Lag order = 7, p-value = 0.01
alternative hypothesis: stationary


        KPSS Test for Level Stationarity

data:  ts_data
KPSS Level = 0.1317, Truncation lag parameter = 5, p-value = 0.1


        KPSS Test for Trend Stationarity

data:  ts_data
KPSS Trend = 0.03015, Truncation lag parameter = 5, p-value = 0.1
```

(b) Stationarity tests for BoxCox TS

Figure 1.3

1.3 Finding a proper model

Plotting the ACF and PACF for the differentiated time series (*Figure* 1.4) suggests that MA(1) might fit well, as the ACF values decay after lag 1.

I follow the procedure described in Course 6 Slide 12 to be sure, fitting ARMA models of order (1,1), (2,2), (3,3) and (4,4) . The order (1,1) shows the most promising results (*Figure* 1.5a), as the AIC scores are similar but the BIC score is the lowest. The next step is to choose between orders (1,0) and (0,1). The latter has marginally better scores (*Figure* 1.5b).

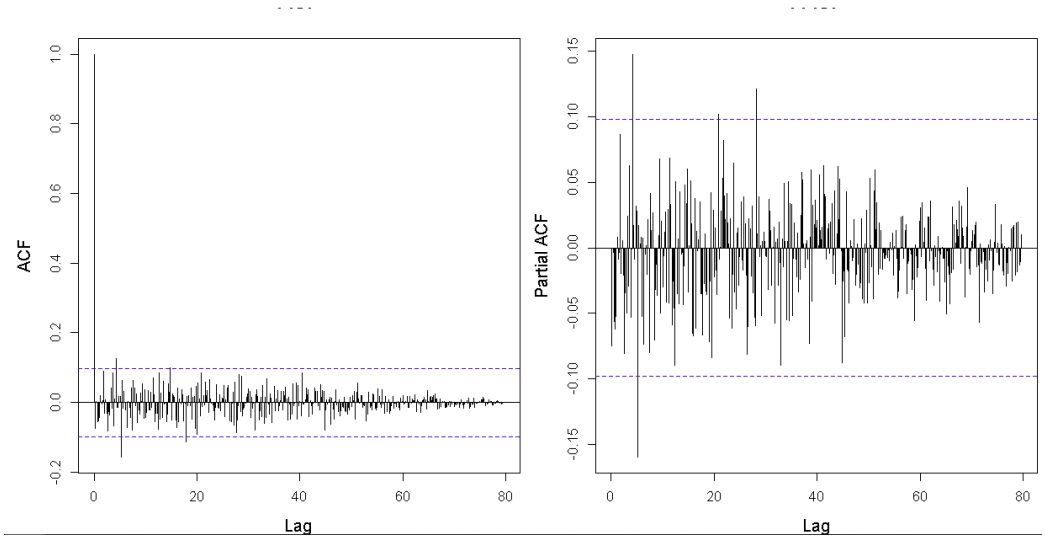


Figure 1.4: ACF and PACF for the diff TS

	model	AIC	BIC	SS
1	(1,0,1)	2064.018	2079.974	10.125
2	(2,0,2)	2066.503	2090.437	10.039
3	(3,0,3)	2064.650	2096.562	9.905
4	(4,0,4)	2063.975	2103.864	9.789

(a) Fitting (p,p) models

	model	AIC	BIC	SS
1	(0,0,1)	2063.638	2075.605	10.166
2	(1,0,0)	2063.661	2075.628	10.167

(b) (1,0) and (0,1) results

Figure 1.5

1.4 Diagnostics

I fit an MA(1) model for the differentiated time series (which means ARIMA(0,1,1) for the original TS) and run diagnostics, as described in Course 6, Slides 14 and 15 (*Figure* 1.6). The residuals are relatively low, the ACF of residuals in the confidence interval and the p values for the Ljung-Box statistic look good too (all greater than 0.05).

The Q-Q plot indicates that the standardized residuals quantiles match the theoretical normal quantiles, which is also a great sign (*Figure* 1.7)

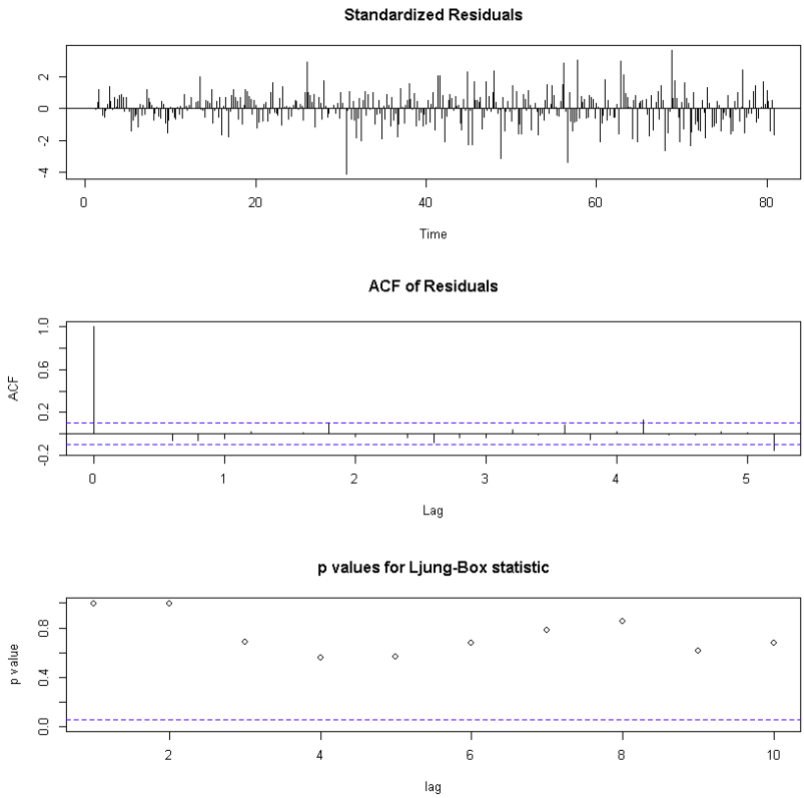


Figure 1.6: Diagnostics for MA(1)

1.5 Forecasting

Finally, it's time for predicting the next 20 values of the time series (*Figure* 1.8). I fitted an ARIMA(0,1,1) model and compared it against an "auto.arima" fitted model from R (which turnede out to be ARIMA(0,1,1) as well). The predictions are ploted together with the actual values from the times series. Now, the predictions are pretty much constant. I think this is due to the fact that the ARIMA(0,1,1) equation is

$$X_t - X_{t-1} \;=\; \varepsilon_t \;+\; \theta\,\varepsilon_{t-1}$$

and the estimated parameters are $\theta = -0.0759$ and $\sigma^2 = 10.16$.

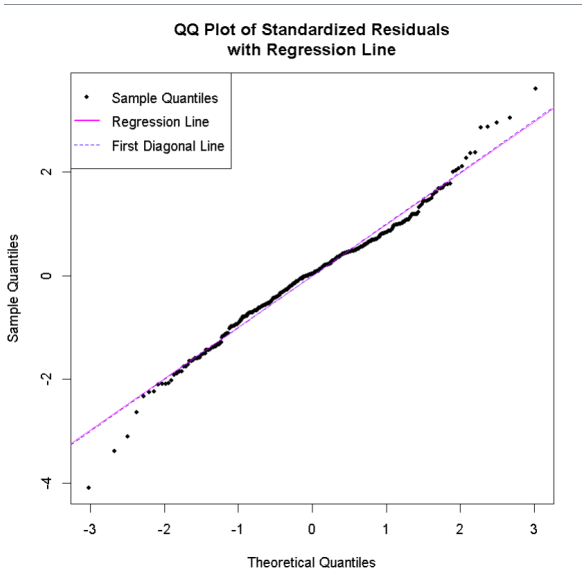


Figure 1.7: Q-Q Plot

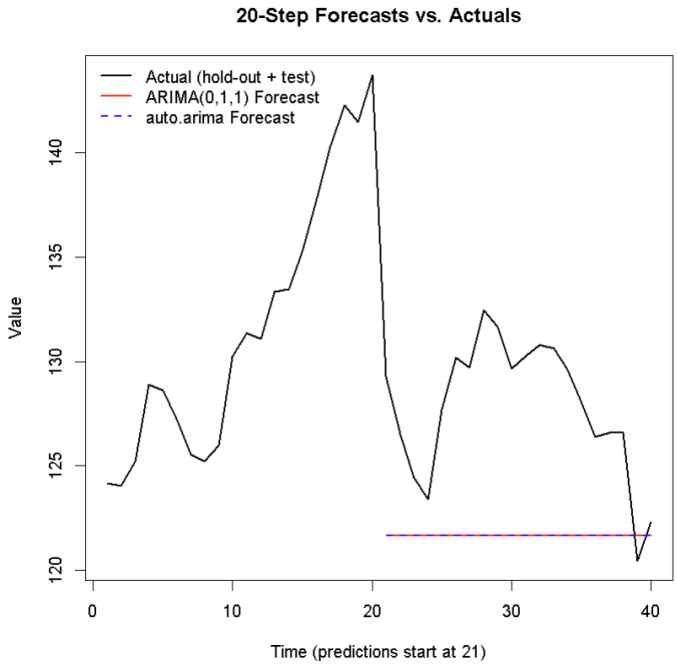


Figure 1.8: Forecast 20 days

Problem 2

For this task, i generated 200 observations from an AR(2) model, stored them in x and after obtaining the desirable results, I generalised for n_{obs} observations from an AR(p_{order}) model. The task itself can be devided into 3 steps:

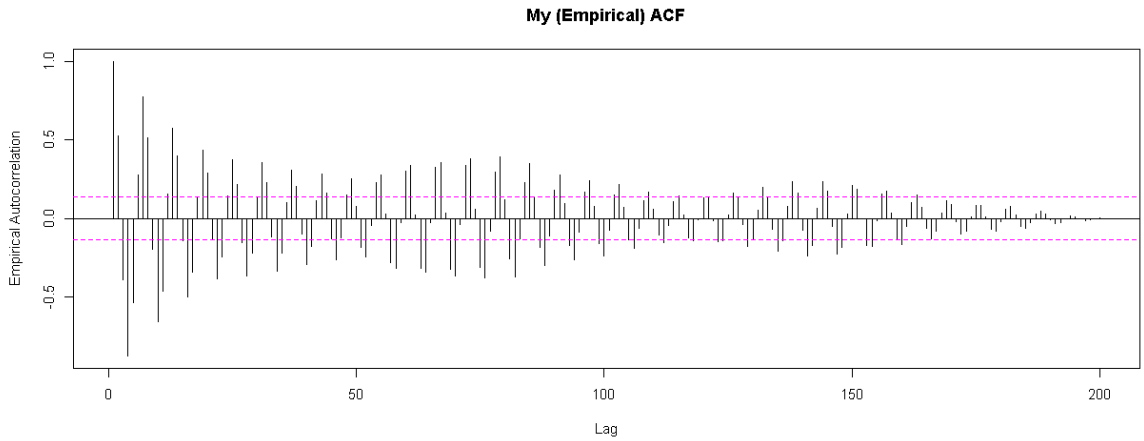
1. Computing the empirical autocovariance
2. Implementation of Dublin–Levinson algorithm
3. Choosing the order of the model

2.1 Empirical Covariance

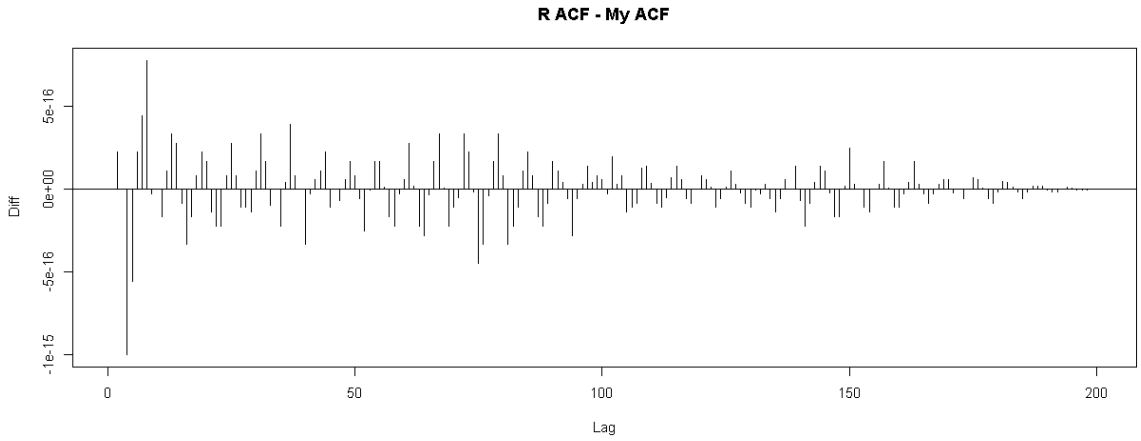
I take inspiration from [1] to compute all the values that I need efficiently. $\bar{x}$ is the mean of the data x and then I compute the deviations vector $dev = x - \bar{x}$. Now i can use it to determine the autocovariance function at lag h :

$$\begin{aligned}\gamma(h) &= \frac{1}{n_{\text{obs}}} \sum_{t=1}^{n_{\text{obs}}-h} \big(x[t] - \bar{x}\big)\big(x[t+h] - \bar{x}\big) \\ &= \frac{\langle \text{dev}[\,h+1\colon n_{\text{obs}}],\, \text{dev}[\,1\colon n_{\text{obs}} - h]\rangle}{n_{\text{obs}}}\end{aligned}$$

I divide the autocovariance values by $\gamma(0)$ and obtain the autocorrelation values(*Figure* 2.1a). The difference between my algorithm and R's algorithm is extremly small (*Figure* 2.1b), so my results are correct.



(a) My ACF



(b) Difference between R ACF and my ACF

Figure 2.1

2.2 Dublin-Levinson

For the Dublin-Levinson algorithm, which will compute the empirical values of the partial autocorrelation function, I am using the equations from Course 4 Slide 26, but [2] Slide 27 helped me make the code more compact, using vectors and a single "for" loop instead of more "for" loops. The plot of my PACF (*Figure* 2.2) values is identical with the one from R.

$$V_0 = \gamma(0),$$

$$\phi_1 = \phi_{1,1}, \qquad \phi_{1,1} = \frac{\gamma(1)}{\gamma(0)},$$

$$\phi_n = \begin{pmatrix} \phi_{n,n-1} \; - \; \phi_{n,n}\,\widetilde{\phi}_{n-1} \\ \phi_{n,n} \end{pmatrix}, \qquad \phi_{n,n} = \frac{\gamma(n) \; - \; \phi'_{n-1}\,\widetilde{\gamma}_{n-1}}{V_{n-1}},$$

$$V_n = V_{n-1} \cdot (1 - \phi_{nn}^2)$$

$$\phi_n = \big(\phi_{n,1}, \ldots, \phi_{n,n}\big)', \qquad \widetilde{\phi}_n = \big(\phi_{n,n}, \ldots, \phi_{n,1}\big)', \qquad \gamma_n = \big(\gamma(1), \ldots, \gamma(n)\big)', \qquad \widetilde{\gamma}_n = \big(\gamma(n), \ldots, \gamma(1)\big)'.$$

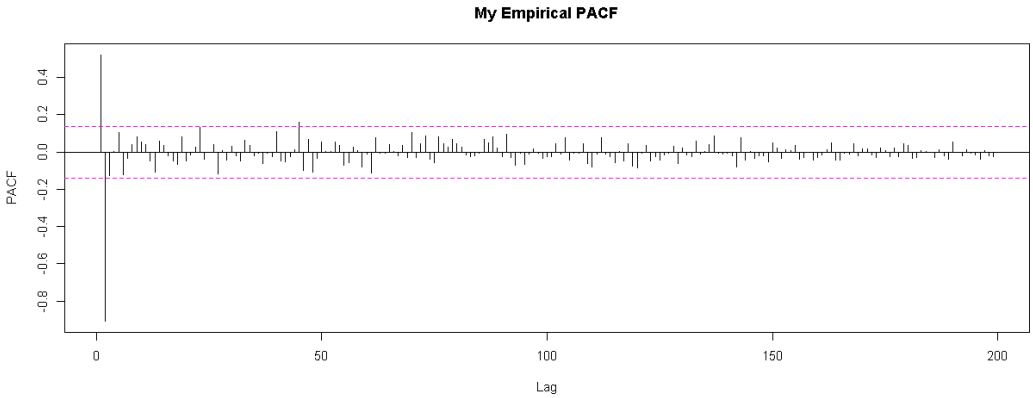


Figure 2.2: My PACF

2.3 Choosing the order of the model

I follow the suggestion for choosing p from Course 5 Slide 9, but I do two more things: I make the confidence interval adjustable ($1 - \alpha$ confidence instead of 0.95) and, whenever I find a p such that $\left|\hat{\phi}_{pp}\right|$ is outside the confidence interval, then I only stop if $\left|\hat{\phi}_{p-1p-1}\right|$ and $\left|\hat{\phi}_{p-2p-2}\right|$ are also outside the confidence interval. This way, I can avoid a random $p = 45$ to be the output of my algorithm, when it should be 2 instead.

Bibliography

[1] Calculating Auto Covariance: Methods, Steps, and Practical Examples,
https://www.numberanalytics.com/blog/calculating-auto-covariance-methods-steps-and-practical-exa

[2] Lecture 9, Introduction to Time Series, Bartlett, Peter L.,Department of Statistics, UC Berkeley
https://www.stat.berkeley.edu/~bartlett/courses/fall2007/lectures/9.pdf

[3] PHLX Gold/Silver Sector (XAU) Historical Data,
https://www.nasdaq.com/market-activity/index/xau/historical?page=1&rows_per_page=10&timeline=y5

[4] https://www.w3schools.com/r/r_data_frames.asp